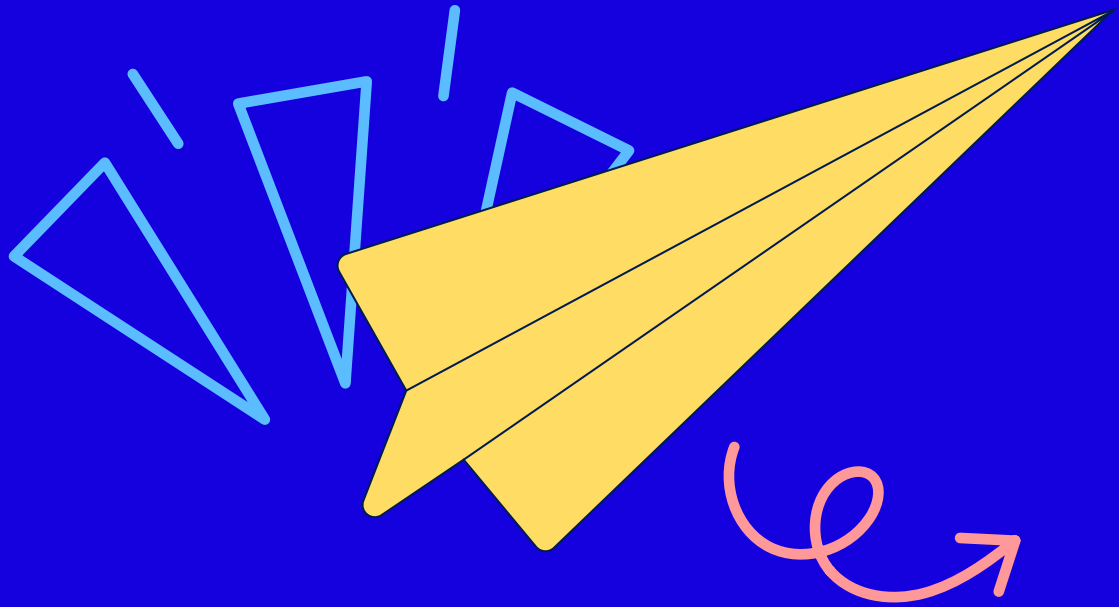
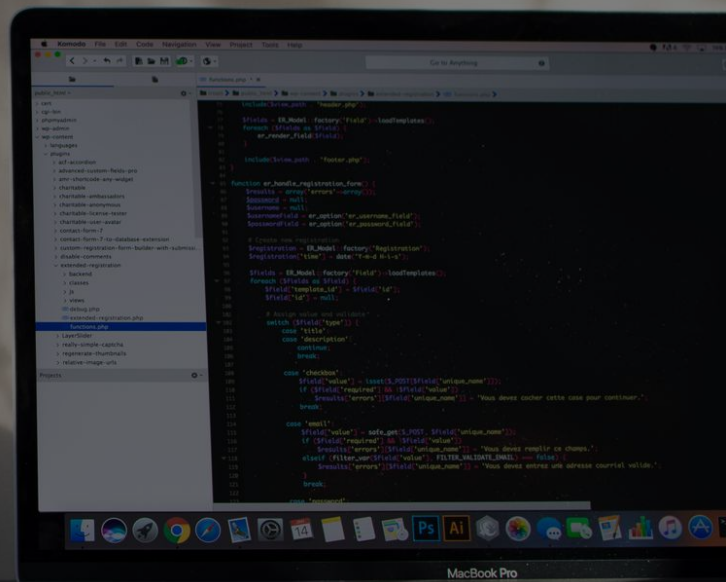




Web components I

Frontend

Web components I



Para comenzar a trabajar con los *web components*, previamente, debes tener los conocimientos suficientes para ser capaz de poder maquetar sitios web complejos, tanto la parte de HTML como la de aportar estilos a través de CSS.

En algún momento, sobre todo, cuando hayas ganado cierta experiencia y hayas hecho unas cuantas maquetaciones, te darás cuenta de que muchas cosas las repites una y otra vez, sabrás qué trozos de HTML los podrías reutilizar en otros sitios web: botones que has evolucionado, formularios, una tarjeta de perfil de usuario, el menú...

Lo cierto es que poder reutilizar esos trozos de código es genial, ¿verdad? Pues esto mismo es lo que nos permiten hacer los web components y el objetivo de este fastbook.

¡Comenzamos!

Autor: Miguel Ángel Muñoz



Qué son los web components



Custom elements



Plantillas HTML



Shadow DOM



React, Angular, Vue, Polymer



¿Qué hemos aprendido?

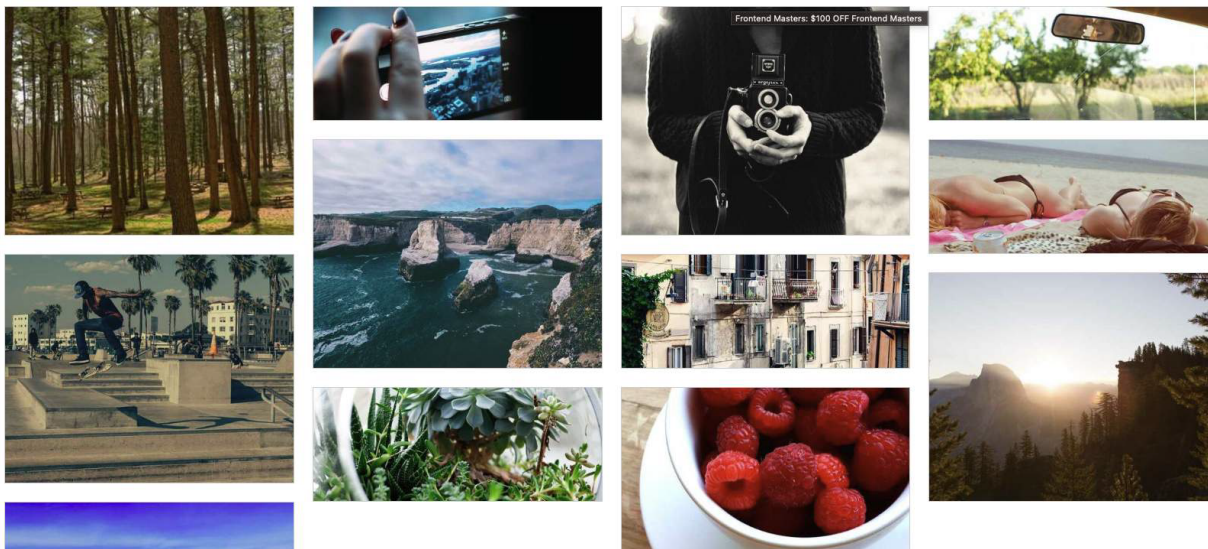
Qué son los web components

Los *web components* son los elementos personalizados que podemos reutilizar importándolos en nuestros sitios web y utilizarlos como si se tratase de cualquier otra tag HTML.

Al igual que utilizamos la tag `<form>`, podríamos utilizar nuestro componente personalizado con el nombre que le otorguemos, por ejemplo, `<my-component>`.

Además, al igual que las tags oficiales, también cuentan con una serie de características especiales. ¿Alguna vez te has planteado de dónde vienen los estilos predefinidos de las tags HTML? ¿O por qué una etiqueta `<button>` se representa así por defecto?

Pues sí, **tienen unos estilos ya aplicados: un borde, un fondo gris, un color de letra.** Ahora, imaginemos que creamos nuestro elemento personalizado y le asignamos unos estilos, por ejemplo, un elemento que nos permita disponer de un conjunto de imágenes en modo *masonry*.



Aquí hemos tenido que aplicar unos cuantos estilos y sería bastante probable que los estilos que vayan asignados a nuestro elemento, si los añadimos como hemos hecho hasta ahora, de alguna forma afecten o se vean afectados por los que existan en el sitio web donde estamos introduciéndolo.

1

Afortunadamente, los web componentes cuentan con la **capacidad de encapsular sus estilos y funcionalidades** de tal forma que no afecten ni se vean afectados por su entorno, a no ser que queramos que así sea.

2

Tenemos la posibilidad de **exponer formas de interactuar** con el *web component* aplicando estilos personalizados o incluso añadiendo HTML dentro y así podremos adaptar el elemento al sitio web en el que lo usemos.

3

Además, tienen la capacidad de poder ser importados en archivos HTML, es decir, no tenemos que copiar y pegar el HTML y los estilos en el sitio web donde los queramos utilizar, simplemente los importamos y empezamos a utilizarlos.

Una maravilla, ¿cierto?

Custom elements

Los elementos customizados son un conjunto de APIs de JavaScript que nos permiten crear nuestros elementos personalizados y definir su comportamiento.

¿Qué son las APIs web?

Las [APIs web](#) son **construcciones que nos permiten crear funcionalidades complejas** de una forma muy simple, es decir, son los navegadores, a través de estas APIs, quienes nos permiten acceder a funcionalidades que ya están desarrolladas y listas para usar.

Veamos algunas de ellas.

FetchAPI

Gracias a ella, podemos conectar con la parte back, con el lado del servidor, para poder obtener datos de la base de datos, por ejemplo, a través de una API de back.

APIs de audio y vídeo —

Nos permiten importar piezas de audio y vídeo y personalizar las interfaces de reproducción.

Web Storage API —

Ideal para almacenar datos en el navegador.

API de geolocalización —

Como su nombre indica, nos permite obtener ubicación del usuario con su previo consentimiento.

Las APIs de JavaScript, en concreto, son **objetos JavaScript que disponen de métodos** para simplificar su uso y cuentan con propiedades que permiten personalizar dicho uso. Por ejemplo, el objeto JavaScript del API de *custom elements* es [CustomElementRegistry](#) que expone **múltiples métodos para su uso**, por ejemplo:

1

CustomElementRegistry.define(): define un nuevo elemento personalizado.

- 2 **CustomElementRegistry.get()**: devuelve el constructor del elemento personalizado que se pase por parámetro.
- 3 **CustomElementRegistry.getName()**: devuelve el nombre del elemento personalizado pasándole el constructor.
- 4 **CustomElementRegistry.upgrade()**: actualiza un elemento personalizado.
- 5 **CustomElementRegistry.whenDefined()**: devuelve una promesa vacía que se resuelve cuando un elemento personalizado se define con el nombre dado.

Un ejemplo de *custom element*

Estudiemos el ejemplo del botón, el cual tiene un estilo asignado de un *padding* de 50px, aunque primero vamos a ver cómo lo representaríamos en HTML y CSS.

```
<style>
  button {
    margin: 50px;
  }
</style>
<button>Button HTML</button>
```

Y ahora, observemos cómo definimos nuestro elemento personalizado utilizando el API de *CustomElements*:

```

<script>
  window.customElements.define('my-button', class MyButton extends HTMLElement {
    constructor() {
      super();
      this.attachShadow({ mode: 'open' });
      this.shadowRoot.innerHTML = `
        <style>
          button {
            margin: 50px;
          }
        </style>
        <button>Mi botón personalizado</button>
      `;
    }
  });
</script>

```

Pues bien, para usarlo es tan sencillo como esto:

```

<my-button></my-button>

```

En el ejemplo he usado más cosas de las que he mencionado, pero no te preocupes que las veremos a continuación.

Observamos que para definir un *web component* necesitamos de JavaScript, por eso, el código de definición del elemento personalizado está dentro de la etiqueta `<script>`. Hemos utilizado el método `customElements.define` ('nombre', class) para definir el elemento personalizado que, a su vez, es un parámetro del objeto `window`, ya que, como hemos mencionado antes, es una API web y por tanto cuelga del objeto `window`.

Debemos definir un nombre y una clase constructora de JavaScript que extienda de la clase [HTMLElement](#) para que las instancias de la clase sean del tipo *HTMLElement* (que es otra API web), permitiendo así que tenga todas las propiedades y características de las que dispone un elemento HTML, como los atributos o los diferentes métodos.

Al extender desde una clase, para que se aplique todo lo que contiene dicha clase, estamos obligados a definir el método constructor y, dentro, a llamar a la función *super()* para que ejecute a su vez el constructor de la clase *HTMLElement*.

A partir de aquí, siempre que usemos *this*, hará referencia a la instancia del objeto del elemento que estamos creando. Esto significa que, si tengo el *web component* varias veces en la misma página, cada uno tendrá su propio *scope* y, por tanto, se comportarán de manera independiente.

Los *web components* se sustentan sobre **tres tecnologías** principalmente:

Custom elements —

Ya los hemos estudiado con el uso del objeto JavaScript *CustomElementRegistry*, el que nos permite definirlo como elemento personalizado y que esté disponible para su uso.

Shadow DOM —

En el ejemplo, puedes ver varias funcionalidades que hacen referencia a él (*attachShadow* y *shadowRoot*). Permite que los *web components* queden encapsulados sin afectar a su entorno.

HTML templates —

Las plantillas HTML ponen a disposición dos elementos, *<template>* y *<slot>*, que nos dan la posibilidad de escribir plantillas de HTML que no se renderizan, pero que sí pueden reutilizarse.

El ciclo de vida

Otra cosa que también debemos tener en cuenta de los elementos personalizados es su ciclo de vida y es que un elemento personalizado no tiene por qué estar todo el tiempo presente en el DOM, se puede **añadir en un momento concreto** o incluso eliminar por medio de JavaScript

Cuando vayamos a **definir un elemento personalizado**, al igual que con cualquier clase JavaScript, debemos distribuir el código correctamente, es decir, no metamos toda la funcionalidad en el constructor como hemos hecho en el ejemplo anterior, ya que esto es únicamente un ejemplo. Lo que está dentro del constructor es lo que se ejecuta sí o sí cuando importamos el script, pero hay partes del código que no vamos a querer ejecutar hasta que añadamos el elemento al DOM.

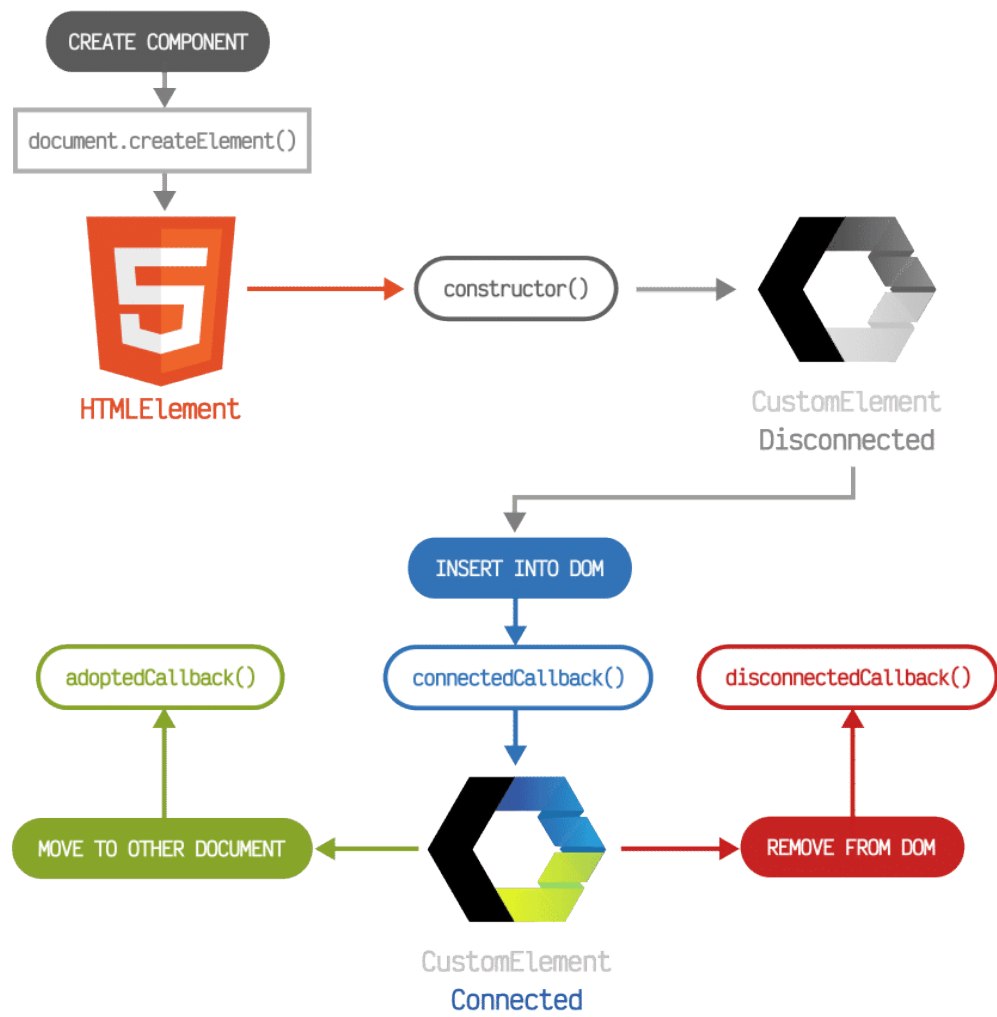
Veamos las llamadas que se producen a lo largo del **ciclo del elemento personalizado**:

CONNECTEDCALLBACK	DISCONNECTEDCALLBACK	ADOPTEDCALLBACK	ATTRIBUTEDCHANGEDCALLBACK
Se llama cuando el elemento es incluido en el DOM por primera vez.			

CONNECTEDCALLBACK	DISCONNECTEDCALLBACK	ADOPTEDCALLBACK	ATTRIBUTEDCHANGEDCALLBACK
Se llama cuando se quita del DOM el elemento personalizado.			

CONNECTEDCALLBACK	DISCONNECTEDCALLBACK	ADOPTEDCALLBACK	ATTRIBUTEDCHANGEDCALLBACK
Cuando el componente se mueve a un nuevo documento.			

CONNECTEDCALLBACK	DISCONNECTEDCALLBACK	ADOPTEDCALLBACK	ATTRIBUTEDCHANGEDCALLBACK
Cuando alguno de los atributos observados del componente personalizado se modifica.			



Fuente: lenguajejs.com.

Plantillas HTML

Las plantillas HTML nos permiten definir estructuras HTML que pueden ser reutilizadas y que por sí mismas no se representan ni tienen ningún efecto.

Cogiendo el ejemplo anterior, podemos utilizar una `<template>` para representar el código que queremos en el web componente y un `<slot>` para poder personalizar el contenido del botón. ¡Veámoslo!

```
<my-button>
  <span slot="button-text">Mi botón con slot</span>
</my-button>

<template id="my-button">
  <style>
    button {
      margin: 50px;
    }
  </style>
  <button>
    <slot name="button-text">Mi botón personalizado</slot>
  </button>
</template>
<script>
  window.customElements.define('my-button', class MyButton extends HTMLElement {
    constructor() {
      super();
      this.attachShadow({ mode: 'open' });
      const template = document.getElementById('my-button');
      this.shadowRoot.appendChild(template.content.cloneNode(true));
    }
  });
</script>
```

¿Qué he hecho? He definido **la estructura del elemento personalizado** dentro del *template*, exactamente igual que como lo tenía antes dentro del constructor, y lo utilizo, dentro del *custom element*, haciendo referencia a la *template* con la siguiente sentencia, usando un selector por ID:

```
const template = document.getElementById('my-button');
```

Después, he incorporado la estructura de esta *template* en el *Shadow DOM* añadiéndole un hijo y clonando el contenido de la *template* a través de la siguiente sentencia:

```
this.shadowRoot.appendChild(template.content.cloneNode(true));
```

En cuanto al slot, nos permite **definir una sección o secciones editables** desde fuera, haciendo que el *web component* sea personalizable. En este caso, he añadido un slot con el nombre *button-text* para que se pueda definir el contenido del botón.

```
<button>
  <slot name="button-text">Mi botón personalizado</slot>
</button>
```

Para ello, usamos la etiqueta `<slot>` y le asignamos un atributo *name*. Además, aunque no es obligatorio, podemos **definir un contenido por defecto**, por si el desarrollador no lo definiese.

A continuación, para definir el contenido que queremos incluir en este *slot*, cuando vayamos a utilizar el *web component*, únicamente tenemos que añadir el atributo *slot* a la etiqueta que tenga que sustituir la etiqueta *slot*, dentro del *web component*.

```
<my-button>  
  <span slot="button-text">Mi botón con slot</span>  
</my-button>
```

Shadow DOM

El [Shadow DOM](#) es la versión en la sombra del DOM, es decir, un DOM oculto que no se ve.

Repasemos ahora qué es el DOM.

De las siglas de los términos ingleses: *document object model*, podemos extraer que es **el objeto que contiene la estructura del documento HTML**, por eso, en JavaScript podemos utilizarlo simplemente poniendo *document*.

 Dispones de más información [aquí](#).

Prestemos atención a este documento HTML:

```

<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Web Components</title>
  </head>
  <body>
    <div>
      <button>Button HTML</button>
    </div>

    <my-button>
      <span slot="button-text">Mi botón con slot</span>
    </my-button>

    <my-button></my-button>

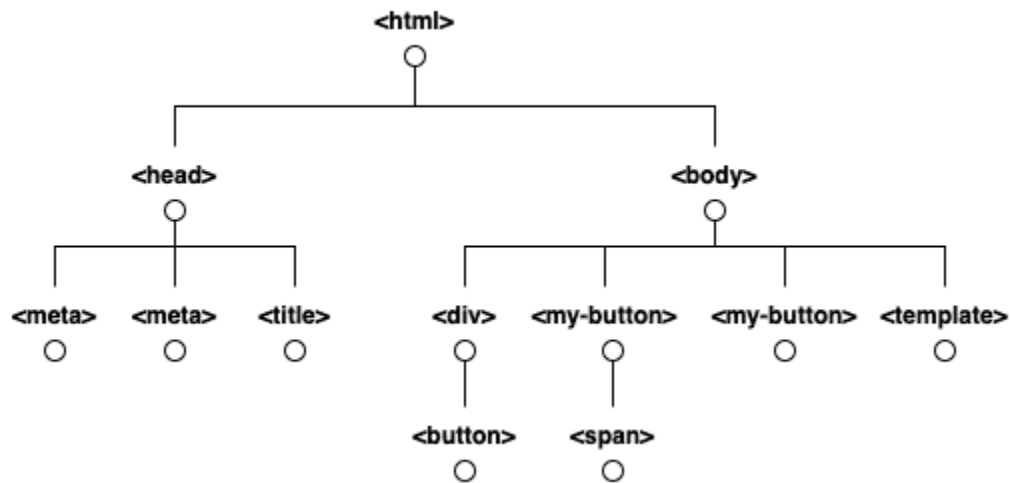
    <template id="my-button">
      <style>
        button {
          margin: 50px;
        }
      </style>
      <button>
        <slot name="button-text">Mi botón personalizado</slot>
      </button>
    </template>
  </body>
</html>

```

Como podemos observar, tiene una **estructura jerarquizada**:

- La etiqueta `<html>` engloba todo el documento.
- La etiqueta `<head>` engloba varias etiquetas como `<meta>` o `<title>`.
- La etiqueta `<body>` engloba el contenido del documento HTML.

Y así sucesivamente, por lo tanto, podríamos representarlo de esta forma:



Cada etiqueta es un elemento y un nodo del DOM que **se representa y se puede visualizar y acceder a él abiertamente**, así como modificarlo globalmente a través de JavaScript o aplicarle estilos a través de CSS. Por eso, si aplico unos estilos a nivel global:

```
button {  
  background-color: red;  
  color: white;  
  padding: 0.5rem 1rem;  
  cursor: pointer;  
}
```

El botón que se representa en el DOM se ve **afectado**:

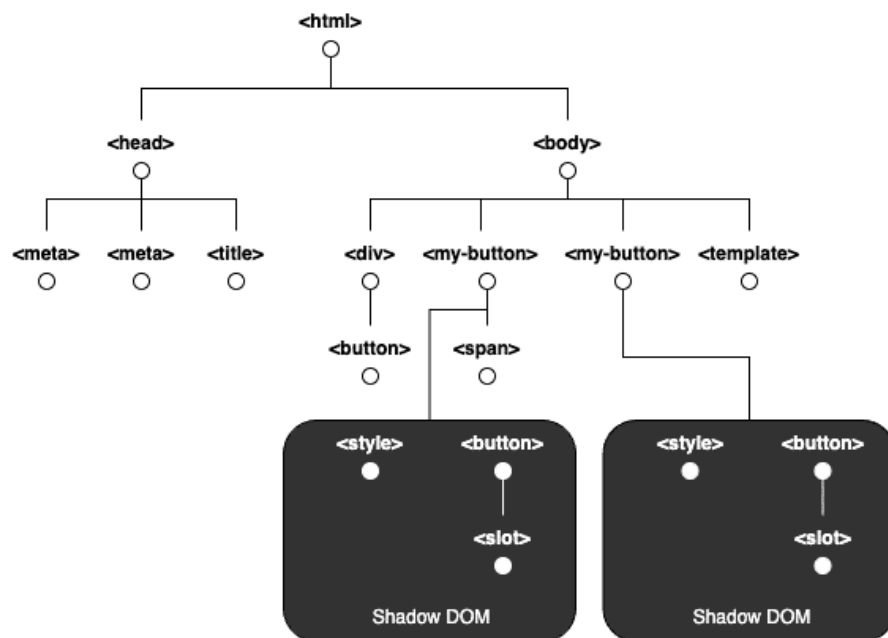
Button HTML

Mi botón con slot

Mi botón personalizado

¿Pero por qué no se representan esos estilos en los botones que tenemos dentro de los *web components*? **La respuesta es simple: por el Shadow DOM.** Es un DOM oculto que se crea enlazado a un nodo del DOM, por lo que es posible crear tantos Shadow DOM como queramos, aunque siempre enlazados a un elemento del DOM.

Siguiendo el ejemplo anterior, podemos **completar el diagrama** representando el Shadow DOM de la siguiente manera:



Como ya hemos visto antes, los estilos globales no afectan al Shadow DOM, ya que está en la sombra, y a la inversa tampoco. Si analizamos el diagrama y el ejemplo previo que hemos visto, vemos que existen unos estilos definidos dentro del *custom element*, pero estos no afectan al exterior. Lógicamente, es posible **permitir la modificación de los estilos del interior** del Shadow DOM, pero hay que exponer atributos para que se asignen valores a los estilos.

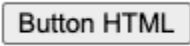
¡Ojo!, el Shadow DOM no es una tecnología exclusiva de los *web components*: puedes utilizarla sin necesidad de estar dentro de un *custom element*, si quieres encapsular una parte del HTML.

De hecho, ya hemos visto cómo crear un nuevo Shadow DOM, basta con **adjuntarlo al elemento del DOM** que queramos.

Fíjate bien:

```
const divElement = document.querySelector('body > div:first-child');
const buttonElement = divElement.querySelector('button');
divElement.attachShadow({ mode: 'open' });
divElement.shadowRoot.appendChild(buttonElement.cloneNode(true));
```

Aquí, hemos cogido el elemento *div*, que tenemos en el DOM, y le hemos añadido un **Shadow DOM en modo abierto**, al que hemos añadido, a través del nodo principal *shadow root* como nodo hijo, una copia del nodo del botón que tenemos dentro del elemento *div*.



Por tanto, tenemos **un nuevo botón dentro de un nuevo Shadow DOM** que hemos creado dentro del *shadow host* (nodo del DOM al que asociamos el Shadow DOM) *div*.

Cabe señalar que el Shadow DOM puede tener **dos valores**:

- **open**: cuando permitimos visualizar lo que hay en su interior.
- **closed**: si queda cerrada la visualización del *tree* del Shadow DOM.

```
▼ <div>
  #shadow-root (closed)
    <button>Button HTML</button>
</div>
```

```
▼ <div>
  ▼ #shadow-root (open)
    |   <button>Button HTML</button>
    |   <button>Button HTML</button>
  </div>
```

Virtual DOM

Quizás hayas escuchado **hablar del Virtual DOM**, sobre todo cuando empiezas a desarrollar con algún framework como VUE o React o en algún artículo sobre los tipos de DOM.

 Si no te suena, puedes leer sobre él [aquí](#).

¿Es lo mismo que el Shadow DOM? —

No, tiene que ver con el DOM, pero no tiene relación con el Shadow DOM. El Virtual DOM es una copia del DOM que se guarda en la memoria para poder realizar los cambios sobre él, sin tener que actualizar el DOM principal y poder transmitir dichos cambios al DOM principal en el momento en el que se considere.

Esto se lleva a cabo con el objetivo de optimizar, ya que la renderización del DOM conlleva trabajo. Por lo tanto, es **mejor tener la posibilidad de aglutinar múltiples cambios** y, posteriormente, transmitirlos al DOM de una sola vez en lugar de transmitir cada cambio por separado.

React, Angular, Vue, Polymer

Igual estás pensando en que siempre que te han hablado o has leído sobre *web components* te los presentaban bajo el contexto de Angular, React, Vue y Polymer.

¿Son lo mismo?

Efectivamente, son lo mismo. De hecho, lo que hemos visto en este fastbook es la base sobre la que se construyen los *web components* de cada uno de los frameworks. Al final, los frameworks, como React, Angular, etc., lo que hacen es poner una capa por encima, en la que tratan de simplificar su creación y añadir funcionalidades por defecto para que se puedan utilizar fácilmente.

Muchos desarrolladores, erróneamente, empiezan aprendiendo algún framework antes de aprender cómo funcionan las cosas en JavaScript, que es **la base de todo** y la forma de entender el por qué y cómo funcionan las cosas.

Es, como se suele decir, empezar la casa por el tejado. Porque si entiendes las bases de cómo se construyen las casas, te será mucho más sencillo e intuitivo entender cómo se han construido determinadas casas (frameworks) e, incluso, podrás construir la tuya propia.

Un desarrollador que entiende los fundamentos es capaz de convertirse en arquitecto y ofrecer soluciones lógicas; mientras que alguien que solo controla frameworks será capaz de desarrollar en ese framework en concreto, pero contará con muchas menos posibilidades de evolucionar.

¿Qué hemos aprendido?

Durante este fastbook hemos visto qué son los *web components* y de qué tecnologías se componen para poder funcionar. Aunque hemos explorado dichas tecnologías asociadas a los *web components*, esto no significa que estén asociadas únicamente a ellos: **podemos utilizarlas en cualquier otra circunstancia.**

Lo más importante es que ahora practiques y juegues con el Shadow DOM para llegar a entenderlo en profundidad, analiza todas las posibilidades que nos aporta...

En definitiva: **completa el contenido de este fastbook** con la lectura de la documentación que te hemos ido enlazando en cada apartado.

Y una vez que tengas los conceptos claros, crea *web components*, pon en práctica todo lo que hemos visto, haz pruebas, pásale parámetros, intenta preparar estilos internos... ¡Practica!

¡Enhorabuena! Fastbook superado